

From Tiny PyTorch Models to Genomic LLM Fine-Tuning

Ikram Ullah

KAUST Bioinformatics Platform

Core claim:

Fine-tuning a genomic LLM is *still* plain PyTorch.

What changes is **scale** and **tooling**, not the underlying mechanics.

Roadmap

1. Tensors → Tokenized DNA
2. `nn.Module` → Large DNA models
3. Devices and `.to(device)`
4. Training loop → `Trainer`
5. Final layer → Classification head
6. LoRA / PEFT → Extra PyTorch modules
7. Why this mapping matters (critical view)

1. Tensors → Tokenized DNA

Before (toy regression)

```
x = torch.linspace(-1, 1, 100).unsqueeze(1)
y = 2 * x + 1 + 0.1 * torch.randn_like(x)
```

You built tensors by hand and passed them into a model.

1. Tensors → Tokenized DNA (continued)

Now (genomic LLM)

```
tokenizer = AutoTokenizer.from_pretrained(
    "RaphaelMourad/Mistral-DNA-v1-17M-hg38",
    model_max_length=200,
    padding_side="right",
    use_fast=True,
    trust_remote_code=True,
)
tokenizer.eos_token = "[EOS]"
tokenizer.pad_token = "[PAD]"

inputs = tokenizer(
    dna_seq,
    return_tensors="pt",
    padding=True,
    truncation=True
).to(device)
```

- `return_tensors="pt"` → outputs PyTorch tensors
- Same tensor semantics you already know

2. nn.Module → Large DNA Models

Before (tiny model)

```
class TinyLinearModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)

model = TinyLinearModel().to(device)
preds = model(x)
```

You define a module and call it.

2. `nn.Module` → Large DNA Models (continued)

Now (transformer)

```
base_model = AutoModel.from_pretrained(  
    "RaphaelMourad/Mistral-DNA-v1-17M-hg38",  
    trust_remote_code=True,  
).to(device)
```

```
with torch.no_grad():  
    outputs = base_model(**inputs)
```

- Still an `nn.Module`
- `base_model(**inputs)` is just a forward pass
- Output = tensors (just larger)

3. Devices and `.to(device)`

Before

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = TinyLinearModel().to(device)
x = x.to(device)
y = y.to(device)
```

Now

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

inputs = tokenizer(..., return_tensors="pt").to(device)
model = AutoModel.from_pretrained(model_name).to(device)
```

Same concept — only scale differs.

4. Training Loop → Hugging Face Trainer

Before (manual loop)

```
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

for xb, yb in dataloader:
    preds = model(xb)
    loss = criterion(preds, yb)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

You control everything explicitly.

4. Training Loop → Trainer (continued)

Now

```
trainer = transformers.Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=val_dataset,  
)  
trainer.train()
```

Trainer automates the same steps you already learned.

5. Final Layer → Classification Head

Before

```
class TinyClassifier(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(2, 16),  
            nn.ReLU(),  
            nn.Linear(16, 2),  
        )
```

Linear → logits

5. Final Layer → Classification Head (continued)

Now

```
model = AutoModelForSequenceClassification.from_pretrained(  
    model_name,  
    num_labels=2,  
)
```

Hugging Face attaches the same kind of classifier head.

7. Why This Mapping Matters

Two goals

1. Avoid black-box thinking
2. Show continuity with PyTorch fundamentals

Final Comment

Mechanistically

You already know most of what you need.

Genomic LLM fine-tuning = PyTorch at scale.